What Is REST?

Explore the core concepts of REST, including its architecture, key terminologies like resources and methods, and features such as statelessness and cacheability. This lesson helps you understand how RESTful services operate over HTTP and how resources are represented and accessed, laying the foundation for developing REST APIs with Python.

What is REST?

Reusability and the distribution of logic to self-contained units have been the driving force behind the development of technologies such as the Distributed Component Object Model (DCOM) and Common Object Request Broker Architecture (CORBA). However, complexity and platform dependency have marred these from becoming standard technologies for creating and consuming distributed and reusable systems. That's why the concept of web services came into prominence. Since they use Extensible Markup Language (XML) and Hypertext Transfer Protocol (HTTP), web services soon became the most used standard for implementing distributed systems that cut across languages, platforms, and technologies. There are two types of web services: SOAP-based and REST-based. This course will focus on developing web services based on REST using Python.

This lesson concentrates on REST, its terminologies, and features.

Ask

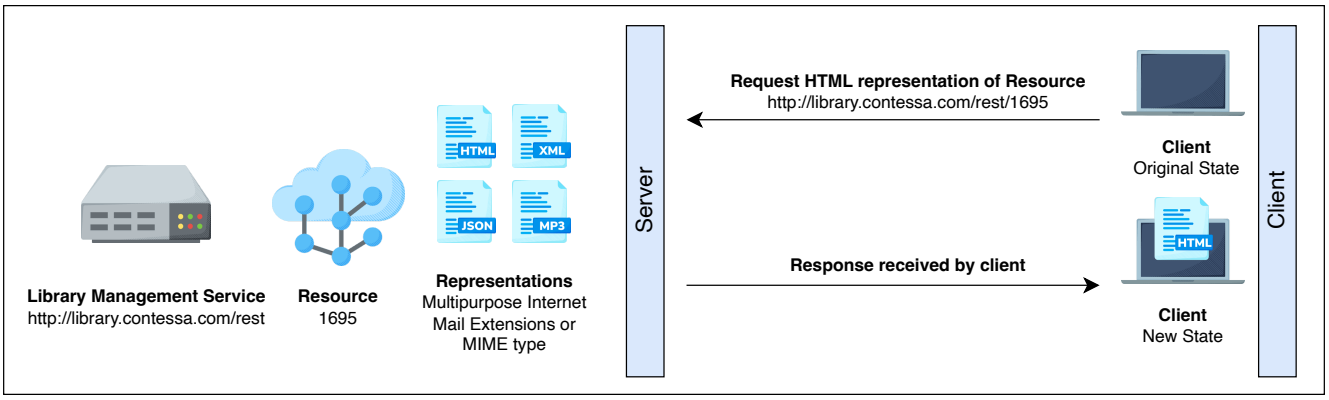
The REST architecture

services that can run over HTTP. Clients developed in different languages can access them just as they access any other web page. In REST, every object accessible via HTTP is a resource, and each of these resources can be accessed via a Uniform Resource Identifier (URI).

If we consider the full form of REST, there are two main parts: representational and state transfer. The former relates to the resource itself, and the latter relates to the client. Let's say, for the sake of example, there is a library management service that can be accessed using the following URI: http://library.contessa.com/rest. It contains resource 1695, which provides details of a particular book. Clients can access it using the URI: http://library.contessa.com/rest/1695. The response is in the form of a HyperText Markup Language (HTML) page. The HTML page is a representation of the resource 1695, which is provided by the service. The resource can have many representations, such as HTML, XML, and JSON. Receiving the response, which in our example is HTML, places the client in one state.

Next, let's say the HTML contains a link to another resource, the details of the author. If the client traverses to the author resource using the link, the client is placed in a different state. So, in short, whenever the client traverses to different resources or a different representation of the same resource, its state is transferred from one representation to another. Hence, the term Representational State Transfer is used for such services that are centered on resources, representations, and state changes.

Ask



REpresentational State Transfer (REST)

REST terminologies

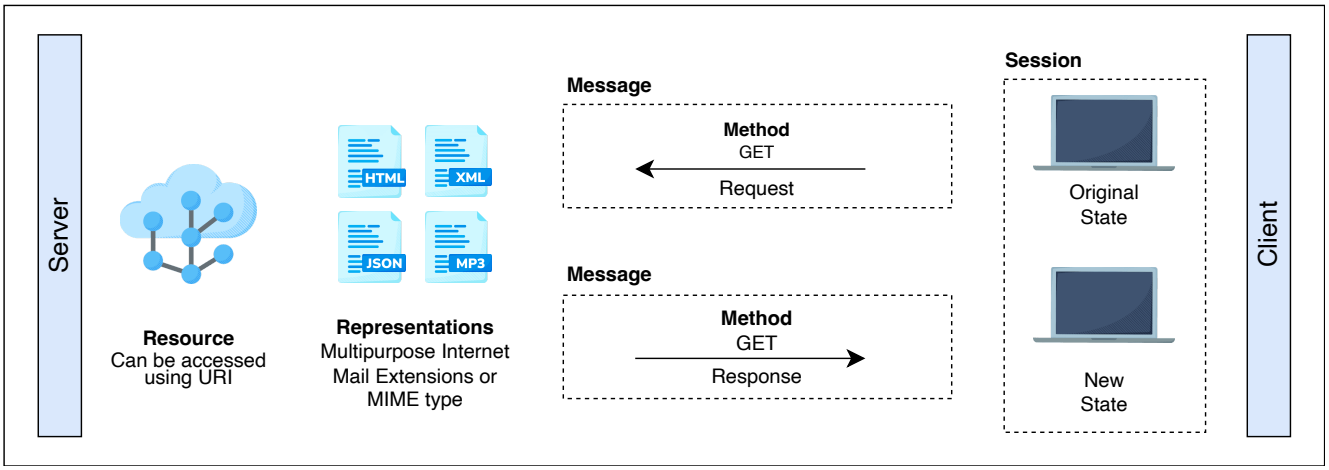
Now that we have the basic concept behind REST, let's look at the most common terminologies used in REST, which are:

- **Resource:** Anything that can be accessed using a URI is termed a resource. Scripts that return records from databases, images, web consumable slides, etc., are examples of resources.
- **Representations:** The formats in which a client can request a resource are known as representations. If a resource can be represented as XML and HTML, then XML and HTML are its representations.
- **Methods:** The way by which a client communicates with the server to perform certain operations is referred to as the method supported by the resource. REST uses HTTP as the protocol for communication, so a resource can support all or a subset of the methods provided by HTTP. For example, in HTTP, we can use GET, POST, PUT, OPTIONS, DELETE, and HEAD methods. It's up to the resource whether or not to support all of the aforementioned methods.
- **Messages:** Each request and response is referred to as a message. One important point to keep in mind is that the messages should be self-contained. For example, a response containing the details of a resource with

Ask

ID 6743 should contain everything related to that resource. Clients shouldn't wait for a second response to have complete data about the same resource.

- **State and session:** The currently sent representation is known as the state of the resource. If the client needs to track what the state was before the current one, they'll need to implement a session at its end. In REST, the server is only concerned with the state of the resource and not the state of the client.



REST terminologies

Features of REST

So, if we want to define the characteristic features of REST on the basis of what we've discussed so far, the features will be:

- It's an architectural style and not a framework or toolkit.
- REST isn't a standard. However, it uses standards for communication, such as HTTP, and for representations, such as XML, JSON, etc.
- It makes use of a pull-based client-server interaction style. The client requests (pulls) a representation of a resource from the server. The se doesn't send the representation unless a client asks the server for the representation.

Ask

- It's stateless. This means the server doesn't keep track of the requests it receives. It's the client's responsibility to provide all the required information in the request.
- The responses are cacheable. The server must mark each response as either cacheable or non-cacheable so that clients can take advantage of caching mechanisms to improve performance.
- The interfaces for a resource in REST must be generic. In other words, all resources must be accessible to clients using the same methods (GET, POST, PUT, DELETE, etc.).
- All the resources must be named using URIs.

Ask